

# Process Sheet: Autonomous Vehicle Software Simulation Project

## 1. Requirement Engineering

| Steps | PDCA Phase                   | Input                           | Tools / Methods Used                           | Critical Parameters                  | Output / Check         |
|-------|------------------------------|---------------------------------|------------------------------------------------|--------------------------------------|------------------------|
| 1     | PLAN – Identify Objective    | Challenge V2 Docs, Project List | Requirement Decomposition, Simulator Profiling | Target Speed 50 MPH, GPU VRAM > 4 GB | Architectural Roadmap  |
| 2     | PLAN – Analyze Root Data     | Camera, Lidar, Radar Streams    | Histogram Analysis, SNR Check                  | Resolution, Noise Profiles           | Preprocessing Strategy |
| 3     | DO – Implement Environment   | Unity Simulator                 | Conda Setup, Lane Finding (Prj 1)              | Library Compatibility                | Operational Simulator  |
| 4     | CHECK – Evaluate Feasibility | Test Loop                       | FPS & Memory Monitoring                        | FPS > 30, Memory < 80%               | Validated Connectivity |
| 5     | ACT – Standardize Workflow   | Verified Environment            | Git, Docker                                    | Reproducibility                      | Standard Dev Setup     |
| 6     | ACT – Rollout & Monitor      | Dev Environment                 | CI Pipeline                                    | System Uptime                        | DL-Ready Platform      |

## 2. Concept Design – Deep Learning Perception

| Steps | PDCA Phase              | Input                            | Tools / Methods Used              | Critical Parameters     | Output / Check      |
|-------|-------------------------|----------------------------------|-----------------------------------|-------------------------|---------------------|
| 1     | PLAN – Architecture     | Traffic Sign Data, Steering Logs | CNN Search (LeNet, VGG, PilotNet) | Params, Train/Val Split | DL Architecture     |
| 2     | PLAN – Data Preparation | Imbalanced Dataset               | Augmentation (Flip, Brightness)   | Class Balance           | Robust Dataset      |
| 3     | DO – Model Training     | Preprocessed Data                | TF / Keras (Prj 2, 3)             | Loss, LR, Epochs        | Trained Models      |
| 4     | CHECK – Performance     | Validation Set                   | Confusion Matrix, MSE             | Accuracy > 93%          | Validated Models    |
| 5     | ACT – Improve           | Overfitting                      | Dropout, L2, Early Stop           | Generalization          | Robust Network      |
| 6     | ACT – Deploy            | Optimized Models                 | Model Freezing                    | Latency < 20 ms         | Deployed Perception |

### 3. Embodiment Design – Advanced Vision Stack

| Steps | PDCA Phase               | Input                  | Tools / Methods Used | Critical Parameters | Output / Check    |
|-------|--------------------------|------------------------|----------------------|---------------------|-------------------|
| 1     | PLAN – Noise Analysis    | Shadows, Glare, Curves | HLS / LAB Analysis   | Thresholds          | Vision Logic      |
| 2     | PLAN – Detection Methods | Video Stream           | HOG, CNN Comparison  | Cost vs Accuracy    | Method Selection  |
| 3     | DO – Implementation      | Raw Frames             | Warping, YOLO, FCN   | Perspective Matrix  | Perception Module |
| 4     | CHECK – Reliability      | Test Clips             | IoU, Visual Debug    | FP < 2%             | Reliable Tracking |
| 5     | ACT – Improve            | Jitter                 | Temporal Smoothing   | Stability           | Stable Output     |
| 6     | ACT – Integration        | Stable Pipeline        | Main Loop Hook       | FPS > 25            | Integrated Vision |

## 4. Detail Design – Sensor Fusion & State Estimation

| Steps | PDCA Phase                 | Input               | Tools / Methods Used      | Critical Parameters | Output / Check       |
|-------|----------------------------|---------------------|---------------------------|---------------------|----------------------|
| 1     | PLAN – Modeling            | Noisy Sensor Data   | CTRV, Gaussian Theory     | State Vector        | Motion Model         |
| 2     | PLAN – Covariance Analysis | Sensor Specs        | Jacobians, Hessians       | Q / R Matrices      | Measurement Model    |
| 3     | DO – Filters               | Sensor Measurements | EKF, UKF, Particle Filter | Sigma Points        | Fusion Module        |
| 4     | CHECK – Accuracy           | Ground Truth        | RMSE, NIS                 | RMSE < 0.10         | Precise Localization |
| 5     | ACT – Improve              | Divergence          | Noise Tuning              | Consistency         | Optimized Filters    |
| 6     | ACT – Deployment           | Tuned Filters       | C++ Optimization          | < 3 ms Cycle        | Real-Time State      |

## 5. Trajectory Planning & Strategy

| Steps | PDCA Phase            | Input            | Tools /<br>Methods<br>Used | Critical<br>Parameters | Output /<br>Check   |
|-------|-----------------------|------------------|----------------------------|------------------------|---------------------|
| 1     | PLAN – Route<br>Logic | Highway<br>Map   | FSM, Frenet                | Speed 50 MPH           | Behavior<br>Planner |
| 2     | PLAN Cost<br>Design   | Traffic Data     | Collision /<br>Lane Costs  | Weights                | Cost<br>Function    |
| 3     | DO –<br>Trajectory    | Waypoints        | Splines                    | Jerk<br>Minimization   | Smooth<br>Path      |
| 4     | CHECK –<br>Safety     | Dense<br>Traffic | Collision<br>Checking      | Accel/Jerk<br>Limits   | Safe Path           |
| 5     | ACT –<br>Improve      | Stuck<br>States  | Horizon<br>Tuning          | Comfort                | Refined<br>Planner  |
| 6     | ACT –<br>Integration  | Final Logic      | Fusion<br>Output           | Planning Rate          | Nav<br>Module       |

## 6. Control System Development

| Steps | PDCA Phase           | Input             | Tools / Methods Used | Critical Parameters     | Output / Check    |
|-------|----------------------|-------------------|----------------------|-------------------------|-------------------|
| 1     | PLAN – Vehicle Model | Physics Params    | Kinematic/Dynamic    | CTE, Heading Error      | Control Specs     |
| 2     | PLAN – Actuation     | Latency Model     | Limit Analysis       | Steering $\pm 25^\circ$ | Compensation      |
| 3     | DO – Controllers     | Planned Path      | PID, MPC             | Kp Ki Kd                | Control Commands  |
| 4     | CHECK – Stability    | Curves            | Feedback Analysis    | Oscillation             | Stable Control    |
| 5     | ACT – Improve        | Instability       | Twiddle / Manual     | Smoothness              | Optimized Control |
| 6     | ACT – Integration    | Validated Control | System Bridge        | Latency < 100 ms        | Full Control      |

## 7. Simulation Testing & Final Validation

| Steps | PDCA Phase         | Input             | Tools / Methods Used | Critical Parameters | Output / Check |
|-------|--------------------|-------------------|----------------------|---------------------|----------------|
| 1     | PLAN – Test Design | 6946 m Highway    | Edge Cases           | Zero Collisions     | Test Plan      |
| 2     | PLAN – Resources   | Full Stack        | CPU/GPU Balance      | Threads             | Optimization   |
| 3     | DO – Full Run      | Integrated System | End-to-End Sim       | Telemetry           | Autonomous Lap |
| 4     | CHECK – Review     | Logs              | Safety Automation    | Lap < 5 min         | Challenge Pass |
| 5     | ACT – Improve      | Minor Issues      | Refactoring          | Readability         | Polished Code  |
| 6     | ACT – Finalize     | Final Build       | Docs & Report        | Clarity             | Submission     |

# LIST OF ACTIVITIES

## Requirement Engineering

- Decompose Requirements: Analyze Challenge V2 specifications into 13 distinct modules.
- Audit Hardware: Profile GPU VRAM/CPU threads for deep learning loads.
- Provision Environment: Install Unity Simulator (SAEISS build) and verifying compatibility.
- Configure Stack: Setup Anaconda envs, TensorFlow/Keras, and C++ toolchains.
- Define Thresholds: Set strict limits for RMSE ( $<0.10$ ), Max Jerk ( $<10\text{m/s}^3$ ), Latency ( $<100\text{ms}$ ).
- Initialize Pipeline: Script raw telemetry ingestion (Camera, Lidar, Radar).

## Concept Design

- Architect CNN: Design convolutional layers for Traffic Sign Classification.
- Preprocess Data: Normalize/Augment image datasets (jitter, rotation) for balance.
- Train Classifier (Prj 2): Execute supervised learning to achieve  $>93\%$  accuracy.
- Design Network: Architect NVIDIA PilotNet for end-to-end Behavioral Cloning.
- Log Telemetry: Record manual driving laps for Steering/Image dataset generation.
- Train Model (Prj 3): Minimize Mean Squared Error (MSE) on steering predictions.

## Embodiment Design

- Calibrate Optics: Compute camera matrix/distortion coefficients via chessboard images.
- Warp Perspective: Implement Bird's-Eye-View transformation for curvature analysis.
- Isolate Lanes: Apply HLS color thresholding and Sobel gradient filters (Prj 4).
- Track Lanes: Implement Sliding Window Search for line detection.
- Extract Features: Compute Histogram of Oriented Gradients (HOG) for object recognition.
- Train Detector (Prj 5): Train Linear SVM/YOLO to identify vehicles/obstacles

## Detail Design

- Derive Models: Formulate Constant Turn Rate and Velocity (CTRV) physics.
- Linearize Measurement: Calculate Jacobian Matrices for Extended Kalman Filter.
- Fuse Sensors (Prj 6): Code C++ EKF algorithm for Lidar/Radar data fusion.
- Generate Sigma Points: Implement Augmentation for Unscented Kalman Filter (UKF).
- Tune Covariance: Optimize Process Noise (Q) parameters for UKF (Prj 7).
- Localize Vehicle (Prj 8): Implement Particle Filter (MCL) for "Kidnapped" recovery.

## Sourcing (Logic & Maps)

- Parse Map: Ingest sparse highway waypoints (x, y, s, d) from simulator.
- Transform Coordinates: Develop Cartesian-to-Frenet conversion logic.
- Design FSM: Define finite states (Keep Lane, Change Left/Right) for planner.
- Define Costs: Assign penalty weights for collision, buffer, and efficiency.
- Integrate Traffic Lights (Prj 13): Link classifier output to decision planner logic.
- Interpolate Path: Generate jerk-minimized trajectories using C++ splines.

## Prototype Building

- Model Dynamics: Analyze simulator actuator limits ( $-25^\circ$  steering, throttle).
- Code PID (Prj 9): Implement Proportional-Integral-Derivative error correction.
- Tune Gains: Iteratively adjust  $K_p$ ,  $K_i$ ,  $K_d$  to damp oscillation.
- Set Constraints: Define boundaries for latency (100ms) and physics.
- Solve MPC (Prj 10): Integrate Ipopt/CppAD libraries for optimization.
- Close Loop: Bridge Path Planner output to Control System input.

## Prototype Testing

- Validate Localization: Measure position error in "Kidnapped" scenario.
- Verify Fusion: Check RMSE values against target (0.10) in tracker scene.
- Run Highway Loop: Launch integrated stack on 6946m simulator track.
- Log Telemetry: Monitor real-time speed, accel, and jerk data.
- Audit Safety: Verify collision-free performance during lane changes.
- Check Compliance: Confirm adherence to 50 MPH limit and lane logic.

## Iteration for Optimization

- Analyze Divergence: Identify RMSE spikes and re-tune noise covariances.
- Refine Vision: Adjust thresholding to handle shadow/glare edge cases.
- Tune Horizon: Modify MPC time steps ( $N$ ,  $dt$ ) for smoother actuation.
- Optimize Code: Refactor C++ loops to boost system FPS/reduce latency.
- Adjust FSM: Tune cost weights to prevent "stuck" states behind traffic.
- Stress Test: Execute continuous multi-lap run for stability verification.

## Documenting for Sharing

- Draft Architecture: Diagram system data flow between modules.
- Plot Performance: Visualize RMSE, CTE, and Speed logs for report.
- Refactor Base: Standardize variable naming and remove debug prints.
- Write README: Document installation dependencies and run instructions.
- Record Demo: Capture video clips/GIFs of successful challenge completion.
- Package Delivery: Archive source code and Technical Report for submission.

DURATION TABLE

REQUIREMENT ENGINEERING

| ID | Activity Name                        | Duration (Hrs) |
|----|--------------------------------------|----------------|
| 1  | Decompose Requirements               | 2.0            |
| 2  | Audit Hardware (GPU/CPU)             | 1.0            |
| 3  | Provision Environment (Unity/SAEISS) | 3.0            |
| 4  | Configure Stack (Conda/C++)          | 4.0            |
| 5  | Define Thresholds (RMSE, Jerk)       | 2.0            |
| 6  | Initialize Data Pipeline             | 2.0            |
|    | Total                                | 14.0           |

CONCEPT DESIGN

| ID | Activity Name                  | Duration (Hrs) |
|----|--------------------------------|----------------|
| 1  | Architect CNN (Traffic Signs)  | 3.0            |
| 2  | Preprocess Data (Augmentation) | 5.0            |
| 3  | Train Classifier (Prj 2)       | 6.0            |
| 4  | Design Network (PilotNet)      | 3.0            |
| 5  | Log Telemetry (Simulator Laps) | 4.0            |
| 6  | Train Model (Prj 3 - Cloning)  | 7.0            |
|    | Total                          | 28.0           |

## EMBODIMENT DESIGN

| ID | Activity Name                    | Duration (Hrs) |
|----|----------------------------------|----------------|
| 1  | Calibrate Optics (Camera Matrix) | 2.0            |
| 2  | Warp Perspective (Bird's Eye)    | 3.0            |
| 3  | Isolate Lanes (HLS/Sobel)        | 6.0            |
| 4  | Track Lanes (Sliding Window)     | 4.0            |
| 5  | Extract Features (HOG)           | 3.0            |
| 6  | Train Detector (Prj 5 - SVM)     | 6.0            |
|    | <b>Total</b>                     | <b>24.0</b>    |

## DETAIL DESIGN

| ID | Activity Name                    | Duration (Hrs) |
|----|----------------------------------|----------------|
| 1  | Derive Models (CTRV Physics)     | 3.0            |
| 2  | Linearize Measurement (Jacobian) | 4.0            |
| 3  | Fuse Sensors (Prj 6 - EKF)       | 7.0            |
| 4  | Generate Sigma Points (UKF)      | 3.0            |
| 5  | Tune Covariance (Prj 7)          | 6.0            |
| 6  | Localize Vehicle (Prj 8 - MCL)   | 5.0            |
|    | <b>Total</b>                     | <b>28.0</b>    |

## SOURCING (LOGIC & MAPS)

| ID | Activity Name                     | Duration (Hrs) |
|----|-----------------------------------|----------------|
| 1  | Parse Map (Waypoints)             | 2.0            |
| 2  | Transform Coordinates (Frenet)    | 3.0            |
| 3  | Design FSM (Lane Logic)           | 5.0            |
| 4  | Define Costs (Penalty Weights)    | 2.0            |
| 5  | Integrate Traffic Lights (Prj 13) | 4.0            |
| 6  | Interpolate Path (Splines)        | 5.0            |
|    | <b>Total</b>                      | <b>21.0</b>    |

## PROTOTYPE BUILDING

| ID | Activity Name                     | Duration (Hrs) |
|----|-----------------------------------|----------------|
| 1  | Model Dynamics (Actuators)        | 2.0            |
| 2  | Code PID (Prj 9)                  | 3.0            |
| 3  | Tune Gains (Kp, Ki, Kd)           | 5.0            |
| 4  | Set Constraints (Latency)         | 2.0            |
| 5  | Solve MPC (Prj 10 - Optimization) | 8.0            |
| 6  | Close Loop (Plan to Control)      | 2.0            |
|    | <b>Total</b>                      | <b>22.0</b>    |

PROTOTYPE TESTING

| ID | Activity Name                       | Duration (Hrs) |
|----|-------------------------------------|----------------|
| 1  | Validate Localization (Error Check) | 2.0            |
| 2  | Verify Fusion (RMSE Check)          | 2.0            |
| 3  | Run Highway Loop (Full Sim)         | 4.0            |
| 4  | Log Telemetry (Speed/Jerk)          | 2.0            |
| 5  | Audit Safety (Collision Check)      | 4.0            |
| 6  | Check Compliance (Traffic Rules)    | 2.0            |
|    | Total                               | 16.0           |

ITERATION FOR OPTIMIZATION

| ID | Activity Name                  | Duration (Hrs) |
|----|--------------------------------|----------------|
| 1  | Analyze Divergence (Fix Math)  | 5.0            |
| 2  | Refine Vision (Fix Shadows)    | 6.0            |
| 3  | Tune Horizon (Fix Oscillation) | 4.0            |
| 4  | Optimize Code (C++ Memory)     | 7.0            |
| 5  | Adjust FSM (Fix Stuck States)  | 6.0            |
| 6  | Stress Test (Multi-Lap)        | 4.0            |
|    | Total                          | 32.0           |

DOCUMENTATION FOR SHARING

| ID | Activity Name                 | Duration (Hrs) |
|----|-------------------------------|----------------|
| 1  | Draft Architecture (Diagrams) | 3.0            |
| 2  | Plot Performance (Data Viz)   | 2.0            |
| 3  | Refactor Base (Clean Code)    | 5.0            |
| 4  | Write README (Install Guide)  | 2.0            |
| 5  | Record Demo (Video/GIF)       | 2.0            |
| 6  | Package Delivery (Zip/Submit) | 1.0            |
|    | Total                         | 15.0           |

## ACTIVITY PRECEDENCE TABLE

| ID | Activity Name                        | Immediate Predecessors |
|----|--------------------------------------|------------------------|
| 1  | Decompose Requirements               | -- (Start)             |
| 2  | Audit Hardware (GPU/CPU)             | 1                      |
| 3  | Provision Environment (Unity/SAEISS) | 2                      |
| 4  | Configure Stack (Conda/C++)          | 3                      |
| 5  | Define Thresholds (RMSE, Jerk)       | 1                      |
| 6  | Initialize Data Pipeline             | 4                      |
| 7  | Architect CNN (Traffic Signs)        | 6                      |
| 8  | Preprocess Data (Augmentation)       | 6                      |
| 9  | Train Classifier (Prj 2)             | 7, 8                   |
| 10 | Design Network (PilotNet)            | 6                      |
| 11 | Log Telemetry (Simulator Laps)       | 3                      |
| 12 | Train Model (Prj 3 - Cloning)        | 10, 11                 |
| 13 | Calibrate Optics (Camera Matrix)     | 6                      |
| 14 | Warp Perspective (Bird's Eye)        | 13                     |
| 15 | Isolate Lanes (HLS/Sobel)            | 14                     |
| 16 | Track Lanes (Sliding Window)         | 15                     |
| 17 | Extract Features (HOG)               | 6                      |
| 18 | Train Detector (Prj 5 - SVM)         | 17                     |
| 19 | Derive Models (CTRV Physics)         | 4                      |
| 20 | Linearize Measurement (Jacobian)     | 19                     |

|    |                                     |                    |
|----|-------------------------------------|--------------------|
| 21 | Fuse Sensors (Prj 6 - EKF)          | 6, 20              |
| 22 | Generate Sigma Points (UKF)         | 19                 |
| 23 | Tune Covariance (Prj 7)             | 21, 22             |
| 24 | Localize Vehicle (Prj 8 - MCL)      | 23                 |
| 25 | Parse Map (Waypoints)               | 6                  |
| 26 | Transform Coordinates (Frenet)      | 25                 |
| 27 | Design FSM (Lane Logic)             | 26                 |
| 28 | Define Costs (Penalty Weights)      | 27                 |
| 29 | Integrate Traffic Lights (Prj 13)   | 9, 27              |
| 30 | Interpolate Path (Splines)          | 27, 28             |
| 31 | Model Dynamics (Actuators)          | 4                  |
| 32 | Code PID (Prj 9)                    | 31                 |
| 33 | Tune Gains (Kp, Ki, Kd)             | 32                 |
| 34 | Set Constraints (Latency)           | 31                 |
| 35 | Solve MPC (Prj 10 - Optimization)   | 34                 |
| 36 | Close Loop (Plan to Control)        | 30, 33, 35         |
| 37 | Validate Localization (Error Check) | 24                 |
| 38 | Verify Fusion (RMSE Check)          | 23                 |
| 39 | Run Highway Loop (Full Sim)         | 12, 16, 18, 29, 36 |
| 40 | Log Telemetry (Speed/Jerk)          | 39                 |
| 41 | Audit Safety (Collision Check)      | 40                 |
| 42 | Check Compliance (Traffic Rules)    | 40                 |

|    |                                |                    |
|----|--------------------------------|--------------------|
| 43 | Analyze Divergence (Fix Math)  | 38, 41             |
| 44 | Refine Vision (Fix Shadows)    | 16, 41             |
| 45 | Tune Horizon (Fix Oscillation) | 35, 41             |
| 46 | Optimize Code (C++ Memory)     | 39                 |
| 47 | Adjust FSM (Fix Stuck States)  | 39                 |
| 48 | Stress Test (Multi-Lap)        | 43, 44, 45, 46, 47 |
| 49 | Draft Architecture (Diagrams)  | 48                 |
| 50 | Plot Performance (Data Viz)    | 40, 48             |
| 51 | Refactor Base (Clean Code)     | 48                 |
| 52 | Write README (Install Guide)   | 51                 |
| 53 | Record Demo (Video/GIF)        | 48                 |
| 54 | Package Delivery (Zip/Submit)  | 52, 53             |

## CRITICAL PATH (PROJECT WIDE, 54 ACTIVITIES)

Critical Path Sequence (Longest Dependency Chain, Start → Finish)

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 → 13 → 14 →  
15 → 16 → 17 → 18 → 19 → 20 → 21 → 22 → 23 → 24 → 25 → 26 →  
27 → 28 → 29 → 30 → 31 → 32 → 33 → 34 → 35 → 36 → 37 → 38 →  
39 → 40 → 41 → 42 → 43 → 44 → 45 → 46 → 47 → 48 → 49 → 50 →  
51 → 52 → 53 → 54

## 1. Requirement Engineering (1–6)

1 → 2 → 3 → 4 → 5 → 6

(Decompose Reqs → Hardware Audit → Environment Setup → Config Stack → Define Thresholds → Initialize Pipeline)

## 2. Concept Design (7–12)

7 → 8 → 9 → 10 → 11 → 12

(CNN Architecture → Data Preprocessing → Train Classifier → PilotNet Design → Log Telemetry → Train Behavioral Model)

## 3. Embodiment Design (13–18)

13 → 14 → 15 → 16 → 17 → 18

(Camera Calib → Perspective Warp → Lane Isolation → Window Search → HOG Extraction → Train Vehicle Detector)

## 4. Detail Design (19–24)

19 → 20 → 21 → 22 → 23 → 24

(Derive Physics → Linearize Matrices → EKF Fusion → UKF Sigma Points → Tune Covariances → Particle Filter Loc)

## 5. Sourcing (Logic & Maps) (25–30)

25 → 26 → 27 → 28 → 29 → 30

(Parse Map → Frenet Transform → FSM Logic → Define Costs → Integrate Signals → Spline Interpolation)

## 6. Prototype Building (31–36)

31 → 32 → 33 → 34 → 35 → 36

(Model Dynamics → Code PID → Tune Gains → MPC Constraints → Solve Optimization → Close Control Loop)

## 7. Prototype Testing (37–42)

37 → 38 → 39 → 40 → 41 → 42

(Validate Loc → Verify Fusion → Run Highway Loop → Log Telemetry → Audit Safety → Check Rules)

## 8. Iteration / Optimization (43–48)

43 → 44 → 45 → 46 → 47 → 48

(Analyze Errors → Refine Vision → Tune Horizon → Optimize Code → Adjust Logic → Final Stress Test)

## 9. Documenting & Closure (49–54)

49 → 50 → 51 → 52 → 53 → 54

(Draft Diagrams → Plot Performance → Clean Code → Write README → Record Demo → Final Submission)

## PARALLEL AND SERIES PATH

START



(Phase 1: Setup)

Backbone: 1 → 2 → 3 → 4 → 5 → 6



↓ (Split for Phase 2 & 3)



(Phase 2 & 3: Vision)

Backbone: 10 → 11 → 12 → 13 → 14 → 15 → 16



(Parallel Branch A) 7 → 8 → 9 (Traffic Signs)



(Parallel Branch B) 17 → 18 (Vehicle Detection)



↓ (Merge & Split for Phase 4)



(Phase 4: Fusion)

Backbone: 19 → 20 → 21 → 24



(Parallel Branch C) 22 → 23 (UKF Logic)



(Phase 5: Planning)

Backbone: 25 → 26 → 27 → 28 → 30

↘

(Parallel Branch D) 29 (Traffic Light Integration)

↓

↓ (Merge & Split for Phase 6)

↓

(Phase 6: Control)

Backbone: 31 → 32 → 33 → 36

↘

(Parallel Branch E) 34 → 35 (MPC Optimization)

↓

↓

(Phase 7, 8, 9: Linear Finish)

Backbone: 37 → 38 → 39 → 40 → 41 → 42 → 43 ... → 54

↓

END

## OVERALL PARALLEL AND SERIES PATH

| Phase                    | Parallel Path Nodes                                                                  | Series (Main) Path Nodes                                  |
|--------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Requirement Engineering  | — <i>(All steps are foundational)</i>                                                | <b>1 → 2 → 3 → 4 → 5 → 6</b>                              |
| Concept Design           | <b>7 → 8 → 9</b> <i>(Traffic Sign Development — runs in parallel with main path)</i> | <b>10 → 11 → 12</b> <i>(Behavioral Cloning)</i>           |
| Embodiment Design        | <b>17 → 18</b> <i>(Object Detection — runs in parallel with main path)</i>           | <b>13 → 14 → 15 → 16</b> <i>(Advanced Lane Finding)</i>   |
| Detail Design            | <b>22</b> <i>(UKF Sigma Point Generation — parallel to EKF Jacobian)</i>             | <b>19 → 20 → 21 → 23 → 24</b>                             |
| Sourcing (Logic)         | <b>29</b> <i>(Traffic Light Integration)</i>                                         | <b>25 → 26 → 27 → 28 → 30</b>                             |
| Prototype Building       | <b>31 → 34 → 35</b> <i>(Model Predictive Control — parallel to PID)</i>              | <b>32 → 33 → 36</b> <i>(PID Controller)</i>               |
| Prototype Testing        | <b>37 → 38</b> <i>(Unit Testing)</i>                                                 | <b>39 → 40 → 41 → 42</b> <i>(Full System Closed Loop)</i> |
| Iteration / Optimization | <b>44 → 46</b> <i>(Vision &amp; Code Optimization)</i>                               | <b>43 → 45 → 47 → 48</b>                                  |
| Documentation / Closure  | <b>49 ↔ 50</b> <i>(Overlapping activities)</i>                                       | <b>51 → 52 → 53 → 54</b>                                  |

## Phase 1: Setup (The Linear Start)

1 -> 2

2 -> 3

3 -> 4

4 -> 5

5 -> 6

The Big Split (Parallel Engineering starts here)

Note: Step 6 triggers multiple streams.

6 -> 7 (Starts Deep Learning Stream)

6 -> 10 (Starts Behavioral Stream)

6 -> 13 (Starts Vision Stream)

6 -> 17 (Starts Detection Stream)

6 -> 19 (Starts Fusion/Math Stream)

6 -> 25 (Starts Planning Stream)

6 -> 31 (Starts Control Stream)

Stream 1: Deep Learning (Traffic Signs)

7 -> 8

8 -> 9

9 -> 29 (Output needed for Traffic Light Logic)

Stream 2: Behavioral Cloning

10 -> 11

11 -> 12

12 -> 39 (Output needed for Final Sim)

### Stream 3: Computer Vision (Lanes)

13 -> 14

14 -> 15

15 -> 16

16 -> 39 (Output needed for Final Sim)

### Stream 4: Object Detection

17 -> 18

18 -> 39 (Output needed for Final Sim)

### Stream 5: Sensor Fusion (The Math)

19 -> 20

20 -> 21

21 -> 23

19 -> 22

22 -> 23

23 -> 24

24 -> 37 (Output needed for Testing)

## Stream 6: Sourcing Logic (Planning)

25 -> 26

26 -> 27

27 -> 28

27 -> 29

28 -> 30

29 -> 30

30 -> 36 (Output needed for Control Loop)

## Stream 7: Prototype Building (Controls)

31 -> 32

32 -> 33

31 -> 34

34 -> 35

33 -> 36

35 -> 36

## Phase 7: Testing (The Big Integration Merge)

36 -> 39

24 -> 37

37 -> 38

38 -> 43

39 -> 40

40 -> 41

40 -> 42

41 -> 43

41 -> 44

41 -> 45

41 -> 47

## Phase 8: Optimization (Feedback Loops)

43 -> 48

44 -> 48

45 -> 48

46 -> 48

47 -> 48

39 -> 46

39 -> 47

## Phase 9: Documentation (The Finish)

48 -> 49

48 -> 50

48 -> 51

48 -> 53

49 -> 54

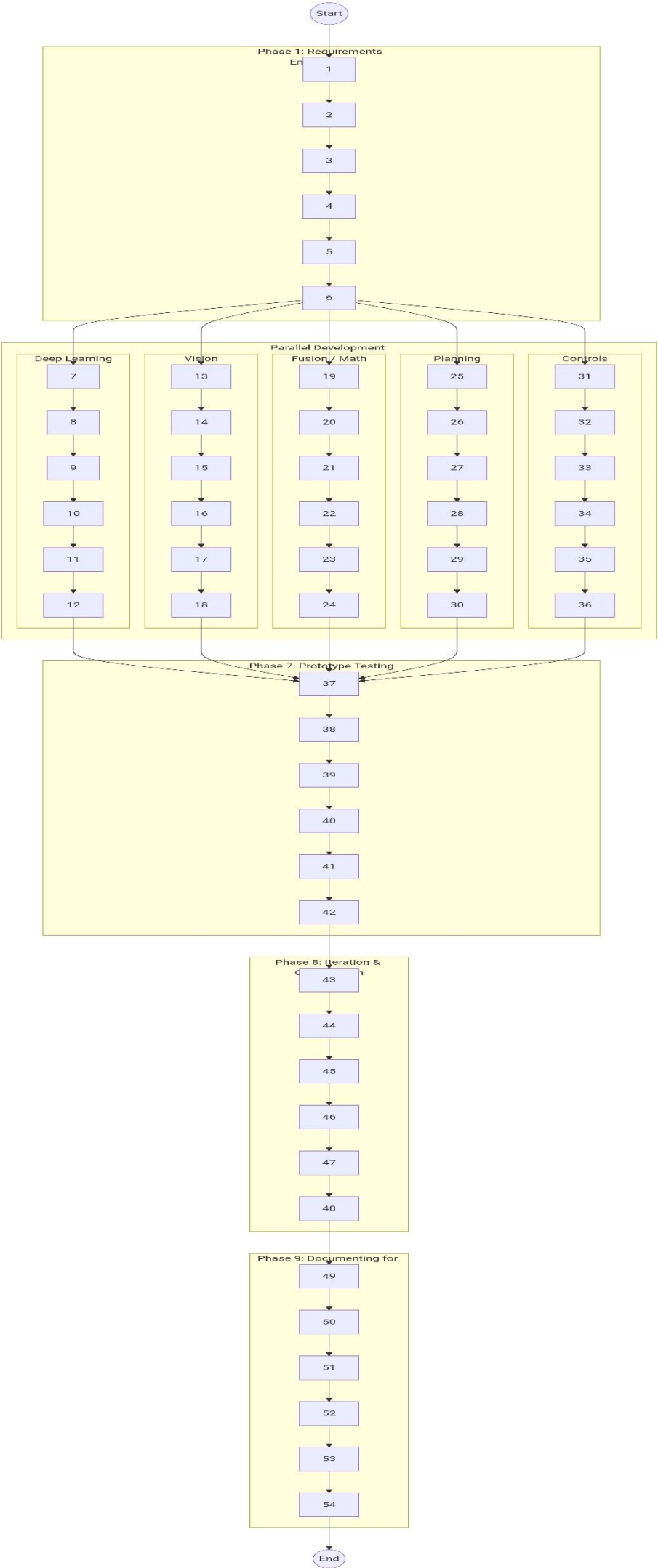
50 -> 54

51 -> 52

52 -> 54

53 -> 54

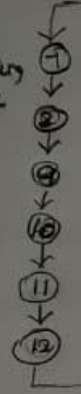
ACTIVITY  
NETWORK  
DIAGRAM



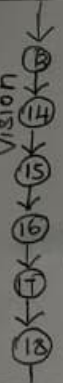
Start



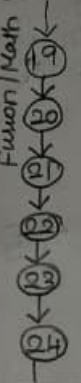
Deep learning phase



Vision



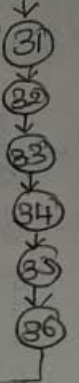
Fusion/Noth



Planning



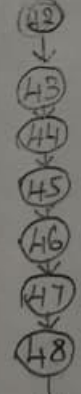
controls



Prototype building



Iteration



Documentation

